

CS 247 Report

Anthony Chen, Christine Li, Elizabeth Xiong

Features that we felt were worthy of bonus consideration have been highlighted.

Introduction

For the CS 247 final project, our group set out to create a complete implementation of RAIINet. The board game as described has a science-fiction theme and is similar in style to [Stratego](#). We took further inspiration from games like [Chess](#) and [Jungi](#) to create a more engaging and innovative experience. We referenced many common design patterns when designing our program and class structure, aiming to follow the SOLID principles. Although we initially encountered challenges concerning scheduling and cross-platform development (different group members utilise Windows, MacOS, and GNU/Linux), we successfully resolved them and have delivered a full memory-safe implementation.

Overview

As discussed in our original plan, the project structure is primarily based on the model-view-controller design pattern. The model represents the game state with the *Game*, *Board*, and *Player* classes. There are two concrete *View* classes for a text-based and graphical view. Finally, the *Controller* runs the game — it holds a *Game* object and processes commands according to the input specification. To initialize the *Controller* instance, the main function parses command-line arguments to let players choose links, abilities, and enable or disable graphics. After initializing, the *Controller* instance processes inputs until the game ends. There are 6 possible input commands:

1. *move*: Controller calls methods on its *Game* to move our piece in the specified direction, end the turn, and display the game board. The logic to validate moves, download links, and resolve battles are separated out in the *Game* class.
2. *abilities*: Controller calls methods on its *Game* to print the abilities of the current player. These abilities were parsed by main, passed to the instantiated Controller, and passed again when Controller instantiates a *Game* object

For a detailed discussion of our unique abilities, please refer to the subsequent section. It suffices to say that we have a separate class for each of the 5 required and 3 unique abilities that all inherit from a virtual *Ability* class. Each concrete instance of an *Ability* subclass keeps track of the player that owns it, as well as how many times it was used.

3. *ability*: Controller reads the desired ability before calling *useAbility()* in the *Game* object. Then the *Game* determines the ability referenced and calls its *use()*. Each ability has a different implementation of *use()* that may request additional input from the user, but all of them interact with the wider state through a reference to the *Game* they are in as well as a pointer to the *Player* the ability belongs to.

A digression into views is presently necessary. We have an abstract *View* class with two concrete subclasses *TestDisplay* and *GraphicDisplay*. Every *Game* has a vector of shared pointers to either *View* type. Which means we can have both *Text* and *Graphics* observers.

4. *board*: Controller calls *printGame()* on its *Game*, which loops through all of its observers and calls *print()* on each of them. Each observer displays information through its format. For example a *TextDisplay* would use helper functions to print each *Player* in the *Game* before printing the *Game* with ASCII characters. A *GraphicDisplay* instead uses helper functions to draw player state, before using helper functions to draw the board state taking into account *Game* information.

Originally, we had a REPL (read-execute-print-loop) structure for our Controller function that deals with player input. However, we soon realised that it would not be able to easily deal with inputs from files. Thus we restructured our Controller to include a command buffer implemented using a stack of shared pointers to input streams. If the command buffer is empty then we try to read more from standard input, and we can handle files.

5. sequence: If we are asked to read commands from a file, then we push the entire sequence of commands to our command buffer. Then the Controller would attempt to consume the command buffer before reading more standard input. Note that this approach also works with nested sequences, as we use a stack data structure.

We dedicated efforts to ensure that our program would be memory-safe. As a start, we completed the project without leaks or explicitly managing our own memory. This was accomplished through heavy use of vectors and smart pointers (particularly shared ones), and we extensively tested the program with Valgrind without finding any leaks. However, it came to our attention that the starter X11 code occasionally leaks memory. We appear to have fixed this by ensuring that worker threads were used to properly update the window.

6. quit: We simply break from our command buffer processing loop and the program exits without leaks. This is because vectors and smart pointers handle freeing memory as needed.

Here are the class relationships in brief. A Controller has a Game. A Game has a Board and tracks associated Players, Links, and Views. A Board has a 2D vector of Cells. Views can register to observe a Board. A Player tracks associated Abilities, Links (owned and downloaded separately), and Views. Views can register to observe a Player. Ability is a virtual superclass with 8 different subclasses each corresponding to an ability.

So what is a View anyway? It is a virtual superclass with `TextDisplay` and `GraphicDisplay` subclasses. Views have `CellState` and `PlayerState` structs are used by subclasses acting as observers on either a Board or Player respectively (since we display separate Board and Player information). Finally, a `GraphicDisplay` has an `Xwindow` object from the starter code. Please see the UML diagram for a full layout and explanation of the above.

There are also more mundane details. We separated each class into header `.h` and implementation `.cc` files and used forward declarations when possible. We have a small but growing test suite that includes both sequence (command) and link (input) files that helps us verify program correctness as well as fix any regressions. We have VSCode and Clang setting files that ensure our group members are using the same formatting options. We have a Makefile to enable easy compilation and cleanup of our program. And we have a README file with instructions on how to execute our project. All this is tracked with the Git version control system, and uploaded to GitHub to facilitate easy collaboration.

Design

We faced many design challenges over this entire project. Here are the 4 most impactful.

1. Our first design challenge came before we had written a single line of code. We found it tough to design a plan of attack and identify class relationships for a game none of us had heard of, much less played. However, we conducted research to relate RAIINet to more familiar board games and our previous domain knowledge became much more helpful. Additionally, we found that in-person collaboration was helpful. Finally, all 3 of us consulted our code for assignment 3. Because there were 3 questions in the assignment, we essentially had 9 different object-oriented programming examples to draw from! This was helpful during the design process.
2. Next, we ran into implementation issues from our original UML diagram. We discovered that there was no clear best way to have one class notify all its

observers. If the Board was responsible for notifying observers, what would happen to the data that was supposed to be private to each player like data and virus locations? It doesn't make sense to have Board own them as then we would need to conduct our own access checks (otherwise either all Players would be able to access Board data or all Players would not be able to access it). On the other hand, it doesn't make sense to just have Players notify Views of changes as some data (ports, firewalls) can apply to both players. In the end we decided to have multiple subjects (Board and Player) and observers (TextDisplay and GraphicDisplay).

3. Another design challenge was how to implement abilities, as different abilities often need to access the Board, Players, Links, and sometimes all of them at once! After analysing other options like the visitor pattern and friend classes, we realised the following framework would enable all the abilities we could think of. So an Ability would have a pointer to the Player that owns them. Finally, the Ability has a use() method that takes a reference to the broader Game and an input stream typically standard. A typical concrete Ability subclass would take input from the user, interact with the Game (e.g. act on the subsequent move), and act on its Player (e.g. download a link). The Player class ensures we have maximum 2 of each Ability.
4. The last major design challenge we ran into was moving a Link. While a basic version is easy to implement, the final version must consider many other factors. The destination might be out of bounds. The target cell might contain a Link already, and the two Links would have to battle (and take Ability effects into account). Next, only certain Links can move into certain squares (server ports). And at the end of this all we have to notify all observers that the old Link should be cleared. There were many subtle bugs to be found. We eventually settled on first validating start and target squares, having the Links battle if necessary (and having the

individual Abilities handle their effects by modifying strengths and the like), and then validating that the final result is a valid state before notifying all observers.

Resilience

In here, we will discuss how our design supports the possibility of various changes to the specification. Several additional scenarios are further discussed in the next section.

What if the board was bigger or smaller, or we had more players?

It would be fairly easy to support these changes. Our Board stores a 2D array of Cells. Currently the dimensions are fixed at 8x8, but we could just as easily create a global variable to accept user input, initialise a correctly-sized board, and later communicate the size of our board to any View instances. We would need to take some care that the board is still big enough to hold all the Links and server ports, however. More players is also easy to implement as long as the Board is big enough to hold all of them, as we currently loop through two instances of Player with id 0 and 1. We could simply add higher id Players.

What if the values of Links were revealed after battles?

Currently, we do not reveal the values of Links after they battle and one is downloaded. This is because we felt this would result in a game with less definite information and more strategy, which would be fun and engaging for players. However, it required some effort to implement. In addition to keeping track of if Links were revealed, we also needed to add setter methods to allow `Game::moveLink` to find the Links involved and do `link->setRevealed()`. Surprisingly, doing this added a lot of complexity to `GraphicDisplay` as we had to take care to redraw more regions of the screen after each move. If the values of Links were revealed after battles then we would be able to remove that complexity.

What if multiple Links could be in the same square?

At first, we implemented Cells to contain a shared pointer to a link. But then we realized that cells technically contain two links at once while they are battling. We decided to instead have a vector of Link shared pointers in each Cell instead. This allows handling battles by moving a Link into the vector, then checking if other Links are in the vector already. Using the dynamic vector allows lots of flexibility as there could be multiple Links coexisting in a Cell, or multiple battles, etc. The downside of using a vector of Links is that accessing and searching are more complex than simply if it was a single Link pointer.

What if we added artificial intelligence to RAIINet?

Recently, developments in Artificial Intelligence (AI) have captured the popular imagination. One equation attributed to Albert Einstein (a famous scientist) is that $E = mc^2 + AI$. This equation means that energy equals mass times the speed of light squared, and the ending term represents AI's potential to change the world. So naturally, we asked ourselves how to add an AI player to our project for users with no friends to play with. We came to the conclusion that this would be hard. This scenario is here to demonstrate that not all changes to our project are easy to make, and software development involves many tradeoffs. Currently, the program is designed around a fixed number of players who alternately enter commands. We would need to substantially change the Controller to either choose commands for an AI player to run with the current program structure, or restructure the Controller to directly make valid moves without going through a command.

How does your program report and handle errors?

In most instances, the last call in the stack—upon failure—prints out a debugging message to cerr and returns a value of false. This lets its caller know that something (typically a low-level operation like an Ability being used or a Link battle comparison, not a high-level operation like moving a single turn) has gone wrong. The caller may either correct the problem and try again, or likewise print to cerr (hopefully with more information) and

return false up the chain. Additionally, we also use try-catch in some crucial parts of the code like the PolarizeAbility to cover both cases of an action (typically for either Player).

Questions

1. We used the model-view-controller design pattern to implement the game. To change our code to instead have 2 displays from each player's point of view, we would most likely add a subclass to View (call it DoubleDisplay) that keeps track of both players' perspectives at the same time. We could then register it as an observer to each Player and the Board just like how TextDisplay and GraphicDisplay. Then every time a View is made or a Player state changes, DoubleDisplay would be notified and it could display the appropriate changes. Alternatively, we could use a TextDisplay and another TextDisplay upside down.
2. We must keep the SOLID design principles in mind, particularly the Open/Closed and Interface Segregation principles. Because abilities (both from the program specification and additional ones added) must be able to affect every aspect of the game, we need to design the game model to provide access methods and methods that can modify model fields. They can be private (and we would later declare abilities as friends) or public methods. Then new abilities (concrete implementations of an abstract ability class) would use setters to get the game model, change the model using getters, and notify the view that the model has changed state (either through a dedicated method or the getters).

Specifically, our general framework that makes adding an Ability easy is a virtual Ability class. We create one subclass for each Ability we want to add. The virtual superclass ensures that all Abilities have a pointer to the Player that owns them, and a use() method that takes a reference to the broader Game and an input stream. So any ability we add can easily take input from the user, interact with the

Game (e.g. only act in a future move), and act on its Player (e.g. give another Link).

This covered all the possible abilities we could think of except for abilities that would affect the View. So we can't easily add an ability blinding the other player.

The 3 unique abilities we added were:

a) Expatriation allows the player to move any one link on the board (including the opponent's links) to any regular board cell (ie., not a server port or firewall, etc).

The letter to specify this ability during setup is E. Example usage: ability 1 B 6 9

b) Stork Visitation allows the player to give birth to a new link onto any regular board cell (ie., not a server port or firewall, etc). The new link has a 50% chance of being either virus or data. The letter to specify this ability during setup is V (avoid name conflict with Scan). Example usage: ability 1 6 9

c) Unsurmountable changes the strength of every piece of the player to the highest strength for the following move. The letter to specify this ability during setup is U. Implementation was interesting as this acts in the future. Example usage: ability 1

3. Great idea! One of the developers fondly recalls playing four-player Stratego as a kid. Speaking generally, developers would most likely need an alternate Game class to handle four players, either by owning an alternate Board or trying to represent four Players with two Board instances. The Controller would also require logic to handle new movement options and to eliminate a Player while letting the other 3 play on. Finally, we would need new code to check if a four-player game has ended.

Although there are many code changes to be made, it would be possible to implement this with minimal additional work given a fully functioning two-player implementation. We would definitely need a new Board class (perhaps a subclass of the old Board) to handle the unique shape. It would need to override moveLink(). Obviously the TextDisplay and GraphicDisplay would also need changes, although we

could minimise the number of changes required by having them display square boards and ask the user to ignore the unused corners themselves. Otherwise we also need to modify Game to initialise up to four players, and to detect when a Game is finally over. Before then, we add functionality to skip moves of defeated Players

Extra Credit

Due to time constraints and following advice from the project guidelines, the developers did not focus on extra credit opportunities, However, we still strongly feel that 3 of our features mentioned above (all highlighted) are worth extra credit consideration.

- Having our project (including the Xwindow code) be completely memory-safe
- Properly handling nested structure commands, enabling nested file commands
- Implementing unsurmountable, an Ability that affects the future and not present
 - This is literally "spooky action at a distance" which impressed Einstein

Reflection

1. Working on this project together taught us many things. One important lesson is that coming up with a clear schedule and list of responsibilities at the start tremendously helps team members split work, even if circumstances later change and the schedule is not followed—the important part is that everyone knows what needs to be done and when the due dates are. We also learned that alternately cloning the code and pushing directly to main works with good communication (the developers had a group chat to ensure that 2 of them were never working on the same piece of code simultaneously). Finally, we learned that having team members easily reachable online to explain their code helped in debugging.

We are reminded of the advice (from Paul Ward, the first professor in our degree to assign us a group project) to never work with your friends in group projects.

Even though we experienced frustrations and challenges working together on our software engineering principles project, we feel that the process worked, we have a well-crafted and functional product, and our friendship is stronger as a result.

2. On group projects, a common response to this sort of question is "we would have started earlier". The grader expects the response and can hardly chastise the group members for having more foresight on their next project. However, we did actually start quite early despite each having many responsibilities—one of us had 4 group projects, another 8 courses, and a third is trying to launch a rocket to 50,000 ft next month. Instead of starting even earlier next time, we think we would put more thought into how we could have this lead to a more enjoyable game for the player instead of simply evaluating potential unique abilities by how technically complex they were to add. Additionally, we would probably budget some time to journey to the student Linux labs and ensure that our program works over SSH from there—just so we can be sure that there will be no issues at the demo.

Conclusion

Overall, the project experience was quite positive. The project developers had the chance to take a board game from real life, modify it, and propose their own additions—who doesn't want to change the rules to win all the time? Additionally, the project was well-scoped and able to be completed by our 3-person group in 2 weeks as promised. There were no significant technical issues with Marmoset, and the specification was quite clear.

Additionally, we were able to gain experience with object-oriented programming and by extension software engineering principles. As we understand it, these are quite in demand skills for the modern workforce. We are proud that throughout this entire project, we

were able to brainstorm ideas, propose a design, and eventually complete a working implementation of the specification. Maybe one day we will have to do this at work!